

PRODUCT GUIDE

bt_air_suite

by NoXoZ.be · v3.0.0

Unified Bluetooth / BLE discovery, inventory, OUI enrichment, HCI capture and authorized Red Team inspection suite for Linux / Kali.

Linux / Kali	Bluetooth / BLE	Offline OUI	Red Team	Wi-Fi workflow parity
--------------	-----------------	-------------	----------	-----------------------

Author	Bruno DELNOZ / @NoXoZ.be
Contact	bruno.delnoz@protonmail.com
Release date	September 24, 2026
Status	Public Release
Repository	github.com/bdelnoz/bt_air_suite

Scope: Designed for owned devices, authorized environments and defensive Bluetooth/BLE investigation. Explicit connection and pairing tests remain separate target actions; disruptive RF jamming and destructive protocol attacks are outside the generic suite.

Contents

1. Overview	4
1.1 What is new in v3.0.0	4
1.2 Who this suite is for	4
2. Installation and prerequisites	4
2.1 Basic installation	4
2.2 Main prerequisites	5
2.3 First-run sequence	5
3. Core concepts and CLI contract	5
3.1 One execution gate, one business action	5
3.2 Control and information actions	6
3.3 Business actions	6
4. Discovery and monitor workflow	6
4.1 One scan slice	6
4.2 Timed monitor sessions	7
4.3 Per-slice post-processing	7
4.4 Kate behavior - intentionally identical to Wi-Fi Air Suite	7
5. Discovery transports and profiles	7
5.1 Transport selection	7
5.2 Profiles	8
6. Device inventory, address types and RSSI	8
6.1 Slice membership	8
6.2 Inventory fields	8
6.3 Why RSSI comes from raw discovery	8
6.4 Known devices versus exclusions	8
7. Offline OUI database and vendor enrichment	8
7.1 Local database	9
7.2 Safe update workflow	9
7.3 Public addresses and random/private BLE addresses	9
7.4 Bluetooth exclusion files	9
8. Post-processing and result artifacts	9
8.1 Result tree	9
8.2 Artifact roles	10
8.3 --nolog	10
8.4 Custom destination	10
9. Authorized Red Team inspection and target actions	10
9.1 Bounded Red Team enrichment	10
9.2 Fingerprint	10
9.3 Service enumeration	10
9.4 Explicit connection test	10

9.5 Explicit pairing test	11
9.6 Explicit boundary	11
10. HCI capture and RSSI monitoring	11
10.1 btmon capture	11
10.2 RSSI monitor	11
11. Controller management and UB500 hotplug	11
11.1 Controller status and power	11
11.2 Secure local controller state	11
11.3 TP-Link UB500 secure-on-attach support	12
12. Repository files and naming	12
12.1 Collision-safe prefixes	12
12.2 Private runtime versus repository content	12
13. Command reference	12
13.1 Discovery and monitor options	13
13.2 Post-process and local data options	13
14. Worked examples	13
14.1 Five-minute Red Team monitor with one-minute slices	13
14.2 Finite duration with remainder	14
14.3 Clean isolated test result directory	14
14.4 BLE-only quick inventory	14
14.5 Refresh OUI then inspect services	14
15. Recommended workflows	14
15.1 Unknown-device investigation	14
15.2 Evidence-oriented acquisition	14
15.3 Low-noise operator workflow	14
16. Troubleshooting	15
17. Frequently asked questions	15
Does --redteam automatically connect or pair?	15
Does an OUI vendor prove who owns the device?	15
Can random BLE addresses be OUI-resolved?	15
What does --nolog remove?	15
What exactly does --open-kate do?	15
Can I use a custom exclusions file?	15
Can I keep an investigation separate from normal results?	16
18. Known limitations and operational boundaries	16
19. Version history	16
19.1 v3.0.0 validation summary	16
20. Project information	16

1. Overview

v3.0.0	Validated v2 baseline	Local files	No telemetry	BlueZ
--------	-----------------------	-------------	--------------	-------

bt_air_suite is a single Bash entry point for Bluetooth and BLE discovery, inventory, post-processing, offline manufacturer enrichment, HCI capture, controller hardening and explicit authorized inspection tests. It is designed to feel operationally familiar to users of **wifi_air_suite.sh**: explicit actions, an execution gate, interval acquisition, per-slice post-processing, predictable result folders and optional asynchronous Kate opening.

The suite uses BlueZ tooling already standard on Linux/Kali. The main discovery engine is `bluetoothctl`; `btmon` is used for HCI traces; `sdptool` is used when available for bounded Classic SDP enumeration.

1.1 What is new in v3.0.0

v3.0.0 is a major release baseline built from the validated v2.0.0 implementation. The operational behavior is deliberately preserved. The major release focuses on a clean, synchronized package and a complete long-form Product Guide.

- Complete Product Guide added to the release package.
- README, INSTALL, CHANGELOG, WHY, EXAMPLES, task specifications and global specifications synchronized to the v3 baseline.
- All supported v2.0.0 actions, options, runtime paths and inventory fields preserved.
- Wi-Fi Air Suite style interval/post-process/Kate workflow preserved exactly.
- Official IEEE OUI update workflow, random-address safety and exclusion enrichment preserved.
- Bounded Red Team SDP enrichment preserved without silently chaining connect or pair operations.

Compatibility: Commands written for v2.0.0 continue to use the same CLI under v3.0.0. The version bump does not rename supported actions or output folders.

1.2 Who this suite is for

Use case	Why bt_air_suite helps
Bluetooth/BLE inventory	Creates per-slice CSV, JSONL and Markdown evidence from actual BlueZ discovery events.
Unknown-device triage	Separates known devices, exclusions, public addresses, private/random addresses and OUI-resolved vendors.
Signal observation	Captures the last RSSI seen in each discovery slice and offers explicit RSSI monitoring for a target.
Protocol visibility	Captures HCI traffic with <code>btmon</code> and enumerates advertised UUID/Classic SDP information.
Authorized Red Team inspection	Adds bounded active SDP enumeration while keeping state-changing connection/pairing tests explicit.
Repeatable operator workflow	Uses timed slices, remainder handling, per-slice post-processing and optional Kate opening like Wi-Fi Air Suite.

2. Installation and prerequisites

The repository is intentionally self-contained: one main Bash script, Markdown documentation, local operator data under `myinfo/`, and runtime output under `.results/`.

2.1 Basic installation

```
chmod +x bt_air_suite.sh
./bt_air_suite.sh --prerequis
./bt_air_suite.sh --doctor
```

The script performs internal sudo authentication only when a selected action requires privileged system changes. Starting the complete script with external sudo is not required.

2.2 Main prerequisites

Tool	Purpose	Typical package/source
bash	Main runtime	Base system
bluetoothctl	BlueZ controller and device operations	bluez
btmon	HCI monitor capture	bluez
rfkill	Bluetooth block-state checks	rfkill / util-linux distro package
sdptool	Optional Classic SDP enumeration	bluez / compatibility tools
curl or wget	Official IEEE OUI database update	curl or wget
timeout	Bounded commands	coreutils
kate	Optional filtered Markdown opening	KDE Kate

2.3 First-run sequence

```
./bt_air_suite.sh --init
./bt_air_suite.sh --prerequis
./bt_air_suite.sh --status
./bt_air_suite.sh --update-oui
./bt_air_suite.sh --simulate --scan --transport le -d 20 --post-process
```

If Bluetooth is soft-blocked or the controller is powered off, use `./bt_air_suite.sh --exec --power-on`. The scan readiness gate prevents misleading result sets when the controller is not usable.

3. Core concepts and CLI contract

3.1 One execution gate, one business action

Operational actions are explicit. Real execution uses `--exec`; dry-run uses `--simulate`. A multi-action invocation is rejected unless the command is a documented control mode.

```
# Real action
./bt_air_suite.sh --exec --scan --transport le -d 30

# Dry-run
./bt_air_suite.sh --simulate --scan --transport le -d 30
```

3.2 Control and information actions

Action	Role
--help / -h	Complete terminal help.
--version	Current release version.
--changelog / -ch	Complete internal append-only changelog.
--prerequis / -pr	Required/optional prerequisite status.
--install / -i	Supported APT prerequisite installation.
--update-oui	Refresh official IEEE OUI database; no --exec required.
--status	Read BlueZ/controller/rfkill state.
--doctor	Paths, prerequisites, controller state and USB candidates.
--init	Create myinfo and runtime layout.
--show-paths	Resolved project/myinfo/runtime paths.
--print-config	Resolved invocation configuration.
--list-results	List runtime result files.
--stop / -st	Stop the active invocation registered by the suite.
--purge / -pu	Purge only managed runtime artifacts; myinfo is preserved.

3.3 Business actions

Action	Purpose
--scan	One bounded discovery slice.
--monitor	One or many discovery slices with finite/infinite session semantics.
--fingerprint	Detailed BlueZ information for an explicit target.
--enum-services	UUID information plus bounded Classic SDP if available.
--capture-btmon	HCI trace in btsnoop format.
--rssi-monitor	Repeated RSSI samples for an explicit target.
--connect-test	Explicit authorized connect/info/disconnect test.
--pair-test	Explicit authorized pairing test, optionally cleaned up.
--secure	Power on, pairable off, discoverable off.
--power-on / --power-off	Controller power management.
--hotplug-install / remove / test	TP-Link UB500-style secure-on-attach support.

4. Discovery and monitor workflow

4.1 One scan slice

A discovery slice uses BlueZ non-interactive timeout semantics so the client remains active for the requested window. The raw output of that exact slice is the evidence source for membership and signal observations.

```
./bt_air_suite.sh --exec --scan \  
  --transport le \  
  --duration 30 \  
  --post-process
```

4.2 Timed monitor sessions

--duration is expressed in seconds. --interval is expressed in minutes, matching Wi-Fi Air Suite. Each interval receives an independent collision-safe prefix.

Command shape	Behavior
--duration 300 --interval 1	5 independent 60-second slices.
--duration 650 --interval 5	300s + 300s + 50s remainder.
--duration 300 without --interval	One 300-second discovery slice.
--infinite --interval 10	10-minute slices until Ctrl-C.
No duration and no interval	Repeated 60-second BlueZ slices until interruption.

4.3 Per-slice post-processing

When --post-process is enabled, each completed slice is processed before the next slice begins. A post-process failure is reported but does not invalidate an otherwise valid acquisition slice.

```
scan slice
-> raw BlueZ evidence
-> per-slice seen-MAC list
-> CSV + JSONL inventory
-> exclusions filter
-> filtered CSV + filtered Markdown
-> enriched artifact
-> generated/ copies
-> optional Kate opening
-> next slice
```

4.4 Kate behavior - intentionally identical to Wi-Fi Air Suite

--open-kate requires --post-process. Every newly generated *.filtered.md is opened immediately after creation using an asynchronous Kate process. The monitor does not wait for the editor to close.

```
./bt_air_suite.sh --exec --monitor \
--redteam \
--duration 300 \
--interval 1 \
--post-process \
--open-kate \
--nolog
```

Important: This is not a final-report-only mode. Five completed intervals can create five filtered Markdown files, and each file is sent to Kate as soon as its own post-process completes.

5. Discovery transports and profiles

5.1 Transport selection

Transport	BlueZ scan mode	Typical use
auto	scan on	Mixed Classic + BLE discovery where supported.
le	scan le	BLE-focused observation.
bredr	scan bredr	Classic BR/EDR-focused observation.

5.2 Profiles

Profile	Aliases	Behavior
passive	--passive	Discovery/capture-oriented workflow; no implicit state-changing target test.
standard	--active	Normal discovery and inspection.
redteam	--redteam / --aggressive-scan	Discovery plus bounded Classic SDP enrichment for devices observed in the slice.

The Red Team profile does **not** silently connect, pair, trust or remove a remote device. Connection and pairing remain separate explicit business actions with an explicit target.

6. Device inventory, address types and RSSI

6.1 Slice membership

BlueZ output contains both local controller events and remote device events. `bt_air_suite` accepts only `Device <MAC>` events as remote membership evidence. `Controller <MAC>` is ignored, preventing the local adapter from appearing as a discovered device.

6.2 Inventory fields

Field	Meaning
timestamp	ISO timestamp for inventory generation.
mac	Normalized remote Bluetooth address.
address_type	public, random-static, random-resolvable, random-non-resolvable, random-reserved or unknown.
oui_prefix	First 24 bits normalized for offline lookup.
vendor	Offline vendor result where attribution is reliable.
oui_status	Resolved / not found / random-not-applicable state.
label / name / alias	BlueZ identification strings.
rss	Last RSSI seen in the exact discovery slice; info fallback only when needed.
tx_power / icon	Optional BlueZ properties.
paired / trusted / connected	Local BlueZ state.
status	Known/unknown classification from local known-device data.
uuids	Advertised/known service UUID summary.

6.3 Why RSSI comes from raw discovery

RSSI is dynamic. A device can publish multiple RSSI updates during a slice and then expose no RSSI at all when queried with `bluetoothctl info` after discovery stops. The suite therefore records the **last RSSI event observed inside the exact raw slice**, with BlueZ info only as a fallback.

6.4 Known devices versus exclusions

A known device remains part of the raw/normal inventory and is marked with status known. An exclusion is removed only from filtered outputs. This preserves evidence while allowing operator-facing reports to focus on devices of interest.

7. Offline OUI database and vendor enrichment

7.1 Local database

The canonical database is `myinfo/oui.txt`. Vendor resolution is performed locally; normal scans do not make a network request per device.

```
./bt_air_suite.sh --update-oui
```

7.2 Safe update workflow

The OUI update action uses HTTPS and a same-directory temporary file. The active database is replaced only after validation succeeds.

- Download the official IEEE OUI text database to a temporary file.
- Validate non-empty content, minimum size and recognizable base-16 entry count.
- Preserve the previous non-empty database as `myinfo/oui.txt.bak`.
- Atomically rename the validated temporary file to `myinfo/oui.txt`.
- Refresh exclusion enrichment after a successful update.
- On download or validation failure, keep the active database unchanged.

7.3 Public addresses and random/private BLE addresses

An OUI prefix is meaningful only when the address can be treated as public. BLE random/private addresses can coincidentally begin with bytes that resemble a registered OUI. The suite classifies address type and avoids asserting a manufacturer for private/random addresses.

OUI is enrichment, not identity proof. Even a correct vendor prefix identifies an address allocation block, not a specific physical owner or unique product model.

7.4 Bluetooth exclusion files

```
myinfo/exclusionsbt.txt  
myinfo/exclusionsbt_enrichi.txt  
myinfo/exclusionsbt_oui_resolved.txt
```

`exclusionsbt.txt` is the operator-maintained input. The other two files are generated/enriched views showing offline OUI context and reliable resolved entries.

8. Post-processing and result artifacts

8.1 Result tree

```
.results/  
  raw/  
  csv/  
  jsonl/  
  filtered/  
  enriched/  
  generated/  
  captures/  
  logs/  
  tmp/  
  archive/
```

8.2 Artifact roles

Directory	Contents
raw/	Raw BlueZ discovery evidence, fingerprints, service reports and Red Team reports.
csv/	Primary structured inventories and RSSI-monitor CSVs.
jsonl/	Machine-friendly one-object-per-line inventory.
filtered/	Exclusion-filtered CSV and Markdown reports.
enriched/	Enriched CSV artifact mirroring Wi-Fi suite layout.
generated/	Convenience copies of generated operator-facing artifacts.
captures/	btmon btsnoop traces.
logs/	Persistent execution logs unless --nolog.
tmp/	Runtime state, seen-MAC lists, PID material.
archive/	Older runtime artifacts moved by --archive-old.

8.3 --nolog

--nolog disables persistent *.log files only. Acquisition data, CSV/JSONL inventories, filtered reports, enriched artifacts and captures remain enabled.

8.4 Custom destination

```
./bt_air_suite.sh --exec --monitor \  
--duration 300 --interval 1 \  
--post-process \  
--dest-dir .results_test
```

This is useful for clean A/B tests or keeping a new release test separate from an older runtime tree.

9. Authorized Red Team inspection and target actions

9.1 Bounded Red Team enrichment

For devices actually observed in a Red Team scan slice, the suite attempts bounded Classic SDP browse when sdptool is available. Each target is time-limited and the number of active probes per slice is capped.

The dedicated report is written under .results/raw/<prefix>.redteam.txt and records target counts and successful SDP results.

9.2 Fingerprint

```
./bt_air_suite.sh --exec --fingerprint \  
--target AA:BB:CC:DD:EE:FF
```

9.3 Service enumeration

```
./bt_air_suite.sh --exec --enum-services \  
--target AA:BB:CC:DD:EE:FF
```

9.4 Explicit connection test

```
./bt_air_suite.sh --exec --connect-test \  
--target AA:BB:CC:DD:EE:FF
```

This action is separate from scan/monitor. It attempts a bounded connection, records BlueZ info and disconnects.

9.5 Explicit pairing test

```
./bt_air_suite.sh --exec --pair-test \  
--target AA:BB:CC:DD:EE:FF \  
--cleanup-pair
```

Pairing changes local pairing state. `--cleanup-pair` requests removal of the local pairing record after the test.

9.6 Explicit boundary

- No RF jamming.
- No forced disruption/deauthentication equivalent.
- No destructive crash testing.
- No aggressive protocol fuzzing.
- No automatic exploit execution.
- The reserved `--lab-destructive` name is explicitly rejected.

10. HCI capture and RSSI monitoring

10.1 btmon capture

```
./bt_air_suite.sh --exec --capture-btmon \  
--controller hci0 \  
--duration 60
```

The suite resolves the requested HCI interface and writes a btsnoop trace under `.results/captures/`. Timed capture uses a bounded timeout and accepts normal timeout/interruption exit codes.

10.2 RSSI monitor

```
./bt_air_suite.sh --exec --rssi-monitor \  
--target AA:BB:CC:DD:EE:FF \  
--duration 60 \  
--rssi-poll 2
```

RSSI monitoring keeps discovery active while repeatedly querying the target. Samples are written to CSV with timestamp, MAC and RSSI. This action is useful when following a known target over a short controlled interval.

11. Controller management and UB500 hotplug

11.1 Controller status and power

```
./bt_air_suite.sh --status  
./bt_air_suite.sh --exec --power-on  
./bt_air_suite.sh --exec --power-off
```

Power-on first clears a Bluetooth rfkill soft block when possible, then asks BlueZ to power the controller on.

11.2 Secure local controller state

```
./bt_air_suite.sh --exec --secure
```

The secure action powers the controller on, then disables pairable and discoverable state.

11.3 TP-Link UB500 secure-on-attach support

```
./bt_air_suite.sh --exec --hotplug-install --ub500-id 2357:0604
./bt_air_suite.sh --hotplug-test
./bt_air_suite.sh --exec --hotplug-remove
```

The hotplug workflow installs/removes the local udev/systemd integration used to apply the secure controller state when the configured USB Bluetooth adapter is attached. The VID:PID is configurable with `--ub500-id`.

12. Repository files and naming

```
bt_air_suite/
  bt_air_suite.sh
  README.md
  INSTALL.md
  CHANGELOG.md
  WHY.md
  EXAMPLES.md
  SPECIFICATIONS.md
  SPECIFICATIONS_GLOBAL.md
  VALIDATION_REPORT.md
  bt_air_suite_v3.0.0_Product_Guide.pdf
  .gitignore
  myinfo/
    known_devices.txt
    exclusionsbt.txt
    exclusionsbt_enrichi.txt
    exclusionsbt_oui_resolved.txt
    oui.txt
  .results/                # runtime, Git-ignored
```

12.1 Collision-safe prefixes

Each acquisition slice receives a fresh timestamp-based prefix with a collision suffix when required. Independent intervals never intentionally overwrite each other.

12.2 Private runtime versus repository content

The repository ships source/documentation and local baseline files. `.results/`, logs, temporary material, archives and local private rule files are excluded by the repository `.gitignore`.

13. Command reference

13.1 Discovery and monitor options

Option	Meaning
--controller auto hciN MAC	Controller selection.
--transport auto le bredr	Discovery transport.
-d / --duration SECONDS	Total action/session duration.
--infinite	Explicit infinite monitor.
--interval MINUTES	Independent monitor slice length.
--profile passive standard redteam	Profile selection.
--passive	Alias for passive profile.
--active	Alias for standard profile.
--redteam	Alias for Red Team profile.
--aggressive-scan	Compatibility alias for Red Team.
--target MAC	Explicit remote target.

13.2 Post-process and local data options

Option	Meaning
--post-process	Filter, Markdown, enrichment and generated copies after each slice.
--no-post-process	Disable post-processing.
--open-kate	Open each new filtered Markdown asynchronously; requires post-process.
--accept	Compatibility/non-interactive acceptance semantics; never bypasses sudo.
--archive-old	Archive existing runtime files once before acquisition.
--nolog / --no-log	Disable persistent log files only.
--dest_dir / --dest-dir DIR	Override runtime result root.
-X / --exclusions-file FILE	Override Bluetooth exclusion list.
--known-file FILE	Override known-device baseline.
--rssi-poll SECONDS	RSSI-monitor sampling interval.
--cleanup-pair	Remove local pairing record after pair-test.
--ub500-id VVVV:PPPP	USB VID:PID for hotplug support.

14. Worked examples

14.1 Five-minute Red Team monitor with one-minute slices

```
./bt_air_suite.sh --exec --monitor \  
  --redteam \  
  --duration 300 \  
  --interval 1 \  
  --post-process \  
  --open-kate \  
  --nolog
```

Result: five independent 60-second slices. Every valid slice is inventoried, filtered and enriched; each new filtered Markdown is opened in Kate without blocking the next slice.

14.2 Finite duration with remainder

```
./bt_air_suite.sh --exec --monitor \  
--duration 650 \  
--interval 5 \  
--post-process
```

Result: 300 seconds + 300 seconds + a final 50-second remainder slice.

14.3 Clean isolated test result directory

```
./bt_air_suite.sh --exec --monitor \  
--duration 300 --interval 1 \  
--post-process --nolog \  
--dest-dir .results_v3_test
```

14.4 BLE-only quick inventory

```
./bt_air_suite.sh --exec --scan \  
--transport le \  
--duration 30 \  
--post-process
```

14.5 Refresh OUI then inspect services

```
./bt_air_suite.sh --update-oui  
./bt_air_suite.sh --exec --enum-services \  
--target AA:BB:CC:DD:EE:FF
```

For every supported action, alias, compatibility option and invalid/reserved example, see the exhaustive **EXAMPLES.md** included in the package.

15. Recommended workflows

15.1 Unknown-device investigation

- Keep your own stable devices in known_devices.txt.
- Keep devices you do not want in operator-facing filtered reports in exclusionsbt.txt.
- Refresh oui.txt periodically with --update-oui.
- Run a finite monitor with interval slices and post-processing.
- Review filtered Markdown files in Kate as each slice completes.
- Use fingerprint or enum-services only when an explicit target needs deeper inspection.

15.2 Evidence-oriented acquisition

- Do not use exclusions as a substitute for raw evidence: raw slice files remain preserved.
- Use a separate --dest-dir for a specific investigation when you want an isolated result tree.
- Keep btmon capture separate from normal inventory when packet-level HCI visibility is needed.
- Record the exact command and release version alongside any exported evidence set.

15.3 Low-noise operator workflow

```
./bt_air_suite.sh --exec --monitor \  
--duration 300 --interval 1 \  
--post-process --open-kate --nolog
```

This keeps the structured results and live review workflow while suppressing persistent runtime logs.

16. Troubleshooting

Symptom	Likely cause	Suggested action
Scan returns immediately	Controller blocked/off or BlueZ discovery failed	Run <code>--status</code> , then <code>--exec --power-on</code> if required. v3 rejects materially early scan windows.
Controller appears as a device	Old pre-fix result set or non-v3 data	Check raw version marker; v3 membership uses Device events only.
RSSI empty in post-scan info	Dynamic RSSI no longer exposed after discovery	Use current CSV/JSONL; v3 takes the last RSSI event from the raw slice.
No vendor for BLE address	Random/private address or OUI not found	Inspect <code>address_type</code> and <code>oui_status</code> ; do not force an OUI assumption.
<code>--open-kate</code> rejected	Missing <code>--post-process</code> or wrong action	Use with <code>--scan/--monitor</code> plus <code>--post-process</code> .
Kate unavailable	Executable not installed/in PATH	Run without <code>--open-kate</code> or install/manage Kate separately.
No SDP data in Red Team report	sdptool absent, device not Classic-SDP reachable, or target does not expose SDP	Check prerequisite and the report status; discovery remains valid.
OUI update fails	Network/HTTP failure or downloaded data failed validation	Existing <code>oui.txt</code> remains unchanged; retry later.
Many old files in <code>.results</code>	Previous runs accumulated	Use <code>--archive-old</code> before a new session, <code>--purge</code> for managed runtime cleanup, or a separate <code>--dest-dir</code> .

17. Frequently asked questions

Does `--redteam` automatically connect or pair?

No. Red Team mode adds bounded Classic SDP enrichment to observed devices. Connection and pairing are separate explicit actions.

Does an OUI vendor prove who owns the device?

No. OUI identifies an address allocation/vendor context. It does not prove a unique product, user or physical identity.

Can random BLE addresses be OUI-resolved?

The suite intentionally avoids claiming a vendor for private/random address types, even if the first bytes resemble a registered prefix.

What does `--nolog` remove?

Persistent log files only. Structured scan/capture results continue to be generated.

What exactly does `--open-kate` do?

After a slice post-process creates its filtered Markdown, the suite launches Kate asynchronously for that specific file. The monitor proceeds immediately to the next slice.

Can I use a custom exclusions file?

Yes. Use `-X FILE` or `--exclusions-file FILE`.

Can I keep an investigation separate from normal results?

Yes. Use `--dest-dir` to point the complete runtime tree at another directory.

18. Known limitations and operational boundaries

- Bluetooth discovery is inherently dependent on remote advertising/discoverability and radio conditions. A device not seen in a slice is not proof that it is absent.
- BLE private/resolvable addresses can change over time; MAC equality or inequality alone is not reliable physical-identity proof.
- Classic SDP visibility varies by device and profile support.
- RSSI is noisy and environment-dependent; use it as a relative signal observation, not a precise distance meter.
- btmon capture reflects controller/HCI visibility available on the local host; it is not an over-the-air spectrum capture.
- The generic suite intentionally excludes RF jamming, forced disruption, destructive crash testing, aggressive fuzzing and automatic exploit execution.

A clean result should be interpreted as: "nothing else was observed by this acquisition under these conditions," not as mathematical proof that no other Bluetooth device exists nearby.

19. Version history

Version	Description
v3.0.0	Major stable baseline from validated v2.0.0 behavior; complete Product Guide and repository-wide documentation/version synchronization.
v2.0.0	Wi-Fi Air Suite style interval/post-process/Kate parity, local OUI update and vendor enrichment, Bluetooth address-type safety, exclusionsbt files and expanded documentation.
v1.1.2	Remote-only Device membership, raw-slice RSSI priority and concrete bounded Red Team SDP enrichment.
v1.1.1	Real scan timing using BlueZ --timeout, per-slice observed membership, controller readiness checks.
v1.1.0	Direct --passive / --active / --redteam aliases.
v1.0.0	Initial unified Bluetooth/BLE suite.

19.1 v3.0.0 validation summary

- Bash syntax check passed.
- No-argument help behavior passed.
- --help, --version and --changelog passed.
- Every long option shown by --help is represented in EXAMPLES.md.
- Packaged IEEE OUI database passed size and entry-count validation.
- Controlled mock scan verified local-controller exclusion, last-RSSI export, post-processing and Kate invocation.

20. Project information

bt_air_suite is an independent NoXoZ.be project for Linux/Kali Bluetooth/BLE investigation and controlled authorized testing.

Item	Value
Author	Bruno DELNOZ

Item	Value
Developer identity	@NoXoZ.be
Email	bruno.delnoz@protonmail.com
Repository	github.com/bdelnoz/bt_air_suite
Current release	v3.0.0
Primary script	bt_air_suite.sh
Exhaustive CLI examples	EXAMPLES.md
Long-form manual	bt_air_suite_v3.0.0_Product_Guide.pdf

© 2026 Bruno DELNOZ / NoXoZ.be - All rights reserved.